HOCHSCHULE
HANNOVER
UNIVERSITY OF
APPLIED SCIENCES
AND ARTS
–
Fakultät IV
Wirtschaft und
Informatik

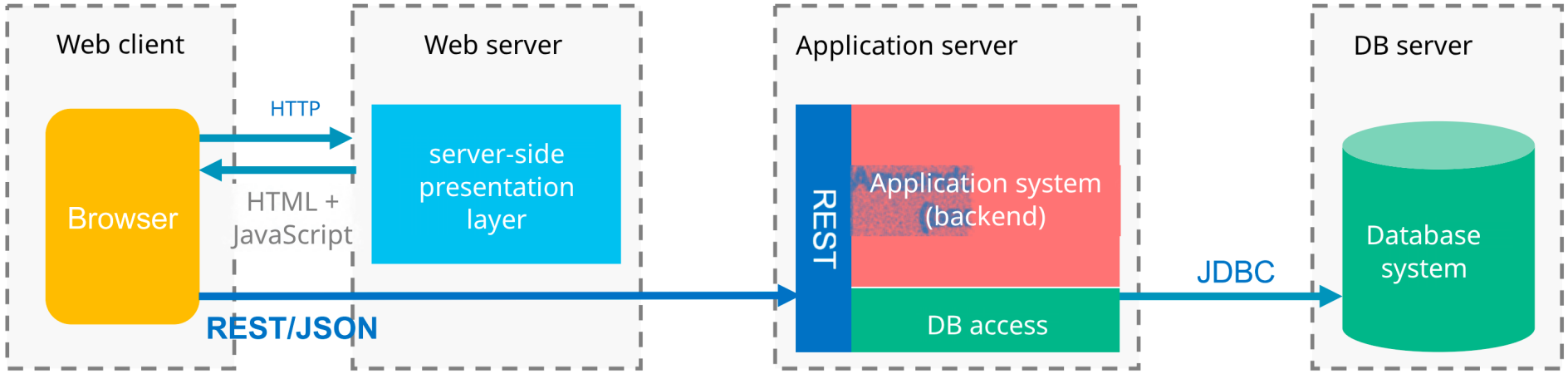
Fortgeschrittene Aspekte verteilter Systeme

Next.js + Spring Boot

Hans Grönniger



Reminder: Client-Side Architecture



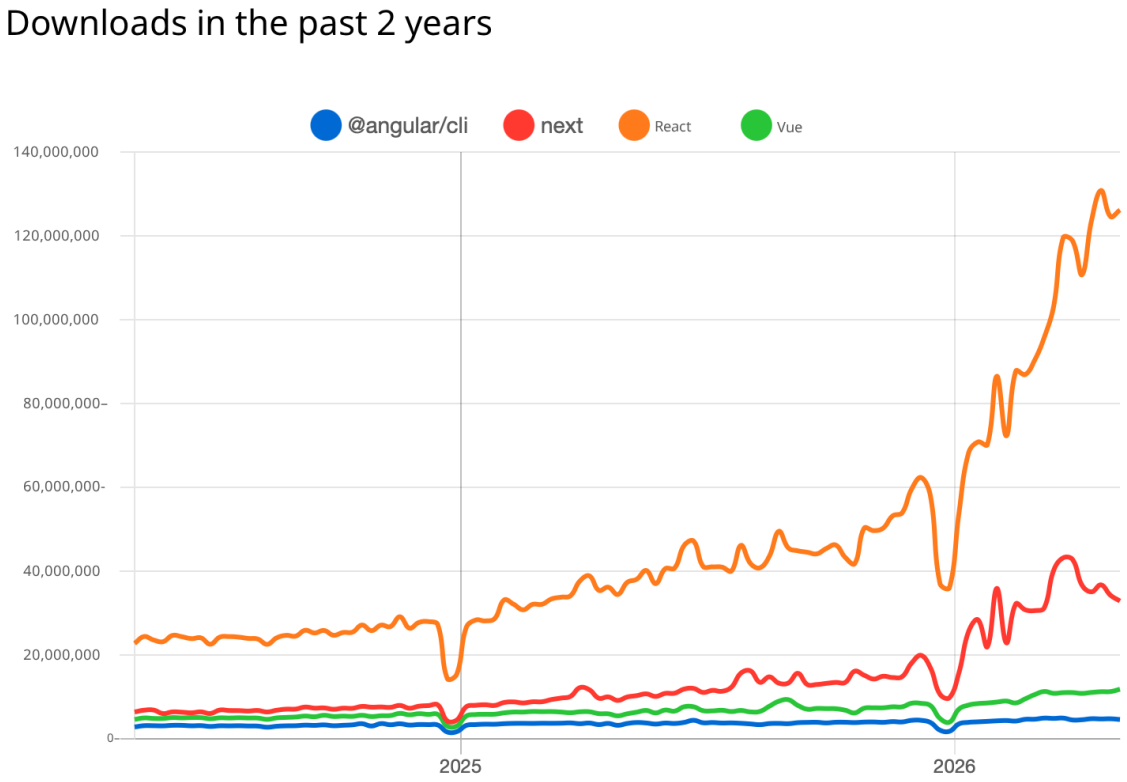
Web server contains web pages with
- static HTML code and JavaScript code

Web client: The browser constructs a complete HTML webpage
- executes the JavaScript code and loads data (usually JSON) from the backend (usually REST)
- and inserts the business data into the HTML page (using JavaScript)

Application server has a REST interface for the web client

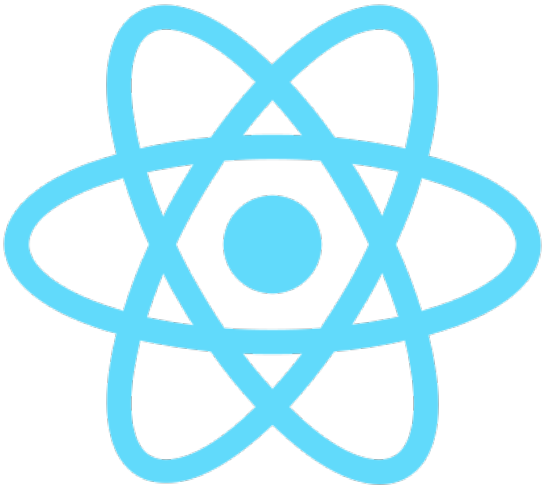
Single-page applications

- Traditional Web:
  - A collection of web pages linked to one another
  - navigation between web pages
  - web pages are reloaded one at a time from the web server
- Web pages may take a long time to load completely; users experience long load times; state must be preserved when reloading another page.
- Therefore single-page application:
  - only one web page is loaded from the web server
  - display of different views
  - Data is stored in the browser, and required resources are reloaded asynchronously



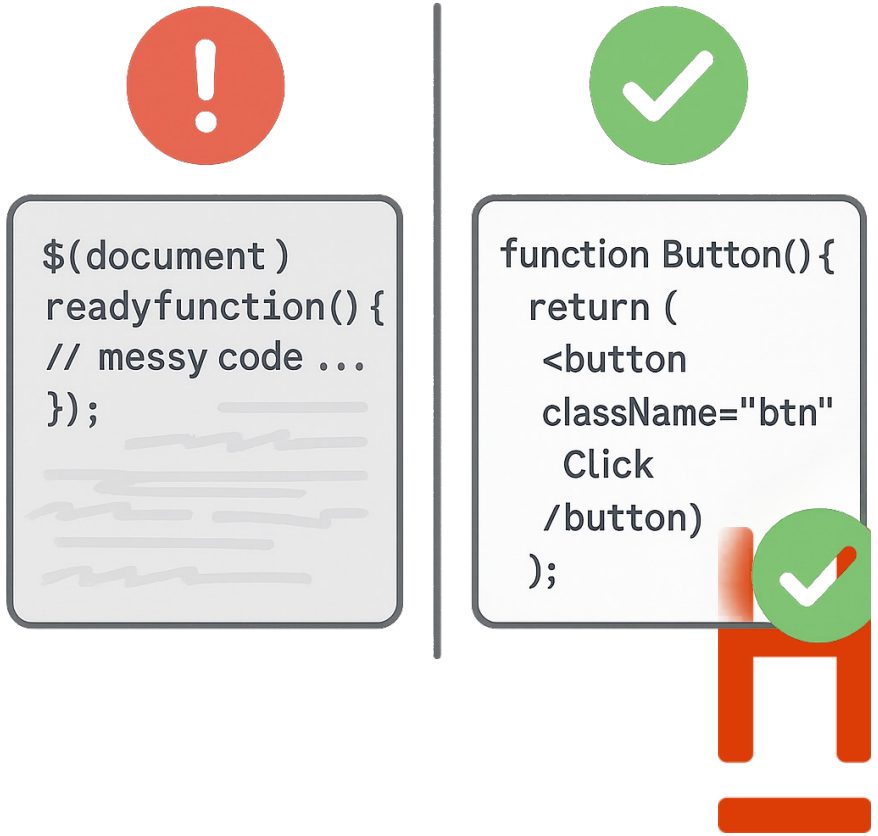
Motivation for React

- **JavaScript UI Development 2010–2012**: Complex web apps were difficult to build with jQuery/DOM manipulation – **Maintenance** and performance suffered.
- Facebook reached its limits with dynamic pages (e.g., the constantly updating News Feed). Changes in one place often broke other code.
- **Goal**: To develop an approach to make **UI complexity** manageable—by breaking the app down into **many small, independent components**.
- The component-based approach should **promote reusability** and reduce side effects (changes in one part do not unexpectedly affect others).



Problem: Manual DOM manipulation

- **Imperative Updates**: In classic JavaScript, the view had to be updated manually (e.g. `document.getElementById(...).innerText = ...`).
- When multiple pieces of data need to be updated, this results in **error-prone code** that is difficult to maintain. Developers must ensure on their own that the UI and data remain consistent.
- **Example**: jQuery code that updates the DOM after an event—becomes hard to follow as the application grows.
- **MVC Challenge**: Controller logic that directly manipulates the DOM violates the separation of presentation and logic (leading to “spaghetti code”).

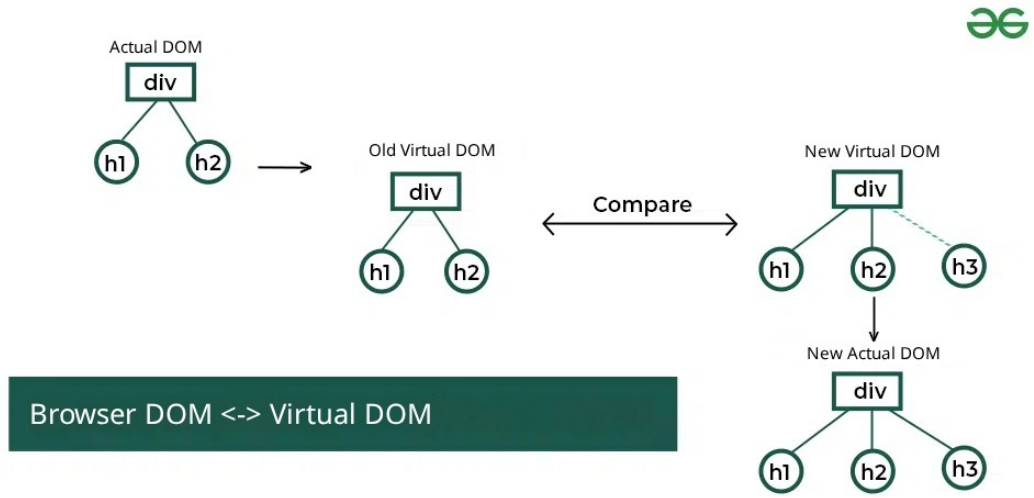


React’s Solution: Declarative UI & Virtual DOM

- **Declarative Rendering**: Developers describe *what the UI state should look like*, not how DOM updates are performed.
  React handles updating
  the view when data changes.

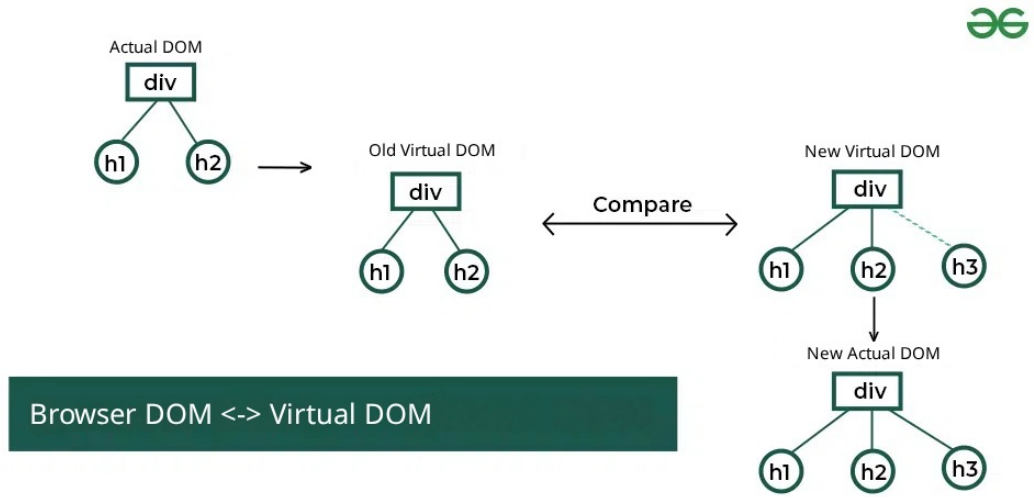
- **Virtual DOM**: React maintains a **lightweight, in-memory representation** of the DOM.
  When the state changes, a virtual
  DOM tree is recalculated and replaces
  the previous one.

- Using an efficient **diff algorithm**, React applies only the minimum necessary
  actual DOM changes. The result: smoother updates, better performance, and less direct DOM manipulation.



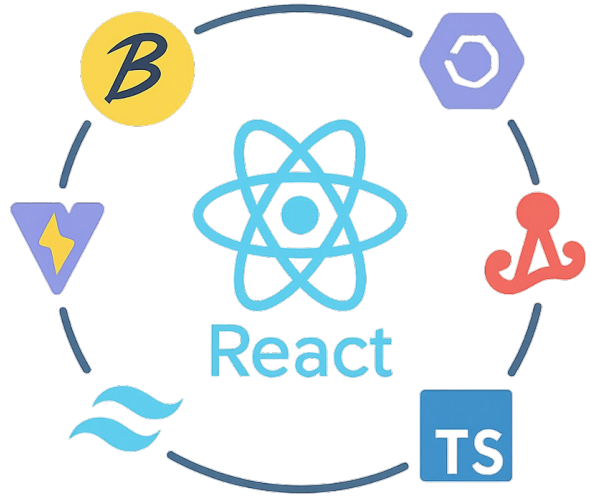
React’s Solution: Declarative UI & Virtual DOM

- **Components**: UI is built using **reusable components**. Each component encapsulates its presentation *and* update logic for a portion of the UI—making it easier to develop large apps.
- **One-Way Data Flow**: In React, data flows in one direction (from top to bottom); this makes it easier to track changes compared to complex two-way binding.



React Ecosystem: Tools & Libraries

- **React alone** provides the basic UI library. To build a complete app, you need additional tools: a build system, routing, state management, etc.
- Typically, **bundlers/compilers** (e.g., *Webpack*, *Vite*) and **transpilers** (*Babel* for JSX/ES6 → ES5) are used
- **Create React App (CRA)** has long been the standard tool for setting up a React SPA (Single-Page Application). It configures the build and dev server, but is focused on purely client-side SPAs.
- **Other libraries**: React Router for routing, Redux or Context API for complex state management, Jest/React Testing Library for testing, etc.
- **Disadvantages of not using a framework**: Developers have to make many decisions on their own (folder structure, code splitting, SEO, SSR)—which can be overwhelming for beginners. This is where Next.js comes into play.





Next.js: Overview

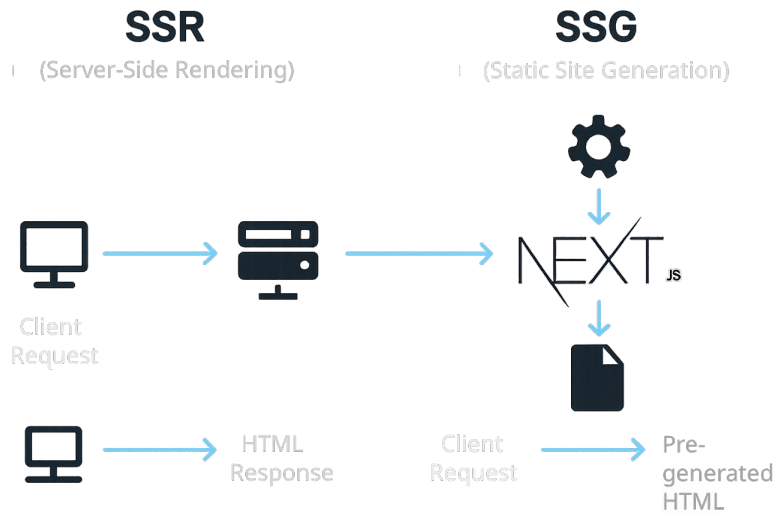
- **Next.js** (developed by Vercel) is a *framework* based on React that provides many necessary features out-of-the-box.
- **Purpose**: To simplify the development of **productive web applications** with React—including routing, server-side rendering, API endpoints, and build optimizations.
- **Philosophy**: “Zero-Config”—Next.js uses proven defaults so that common requirements (code splitting, hot module replacement, routing) work without additional configuration.
- **Comparison to Angular**: Next.js + React is more like a *complete* framework like Angular. React on its own is minimalistic – „just a libaray“, but Next.js provides a structural framework similar to Angular’s CLI+Modules, yet more flexible and less prescriptive.





Next.js: Key Features

- **File-Based Router**: Pages are defined by files/directories—e.g., `app/index.tsx` for the homepage. No manual route setup is required.
- **Server-Side Rendering (SSR)** and **Static Site Generation (SSG)**: Next can render pages *server-side* (for SEO and performance). Developers choose per page: precompile during the build or render on request.
- **API Routes**: Backend logic is possible via `app/api/*` – Next.js thus serves as a full-stack framework (simple REST endpoints without a separate server, often used in BFF pattern).
- **Development Tools**: Built-in dev server (next dev), hot reloading, out-of-the-box TypeScript support. All of this makes Next.js a **versatile choice for modern web apps**



HOCHSCHULE
HANNOVER
UNIVERSITY OF
APPLIED SCIENCES
AND ARTS
–
Fakultät IV
Wirtschaft und
Informatik

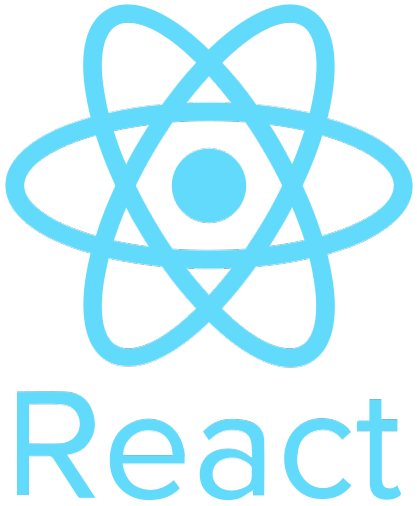
Fortgeschrittene Aspekte verteilter Systeme

React - Basics



Components in React

- **Components** are the building blocks of every React app. They encapsulate a portion of the user interface (UI) as well as the associated logic.
- Every React component is either a **function** or a class (today, mostly function components with hooks). It receives inputs (**props**) and returns a rendering (JSX elements).
- Components make it possible to break down the UI into **independent, reusable parts** . You can examine and develop each part in isolation—which improves maintainability.





Components in React

- Our first component
  - Display of personal data (static, to be fetched from a web service later)

```
const persons = [
    {
        "id": 1,
        "firstName": "Max",
        "lastName": "Muster",
        "position": "CTO"
    },
    …
];
```

```
function List() {
    return (
        <table>
            <thead>
                <tr>
                    <th>ID</th>
                    <th>First Name</th>
                    <th>Last Name</th>
                    <th>Email</th>
                </tr>
            </thead>
            <tbody>
                {persons.map((person) => (
                    <tr key={person.id}>
                        <td>{person.id}</td>
                        <td>{person.firstName}</td>
                        <td>{person.lastName}</td>
                        <td>{person.position}</td>
                    </tr>
                ))}
            </tbody>
        </table>
    );
}
export default List;
```



JSX: JavaScript XML

- **JSX** is a syntax extension for JavaScript that looks like HTML/XML within JS.
  Example: `<Welcome name="Sara" />` in JavaScript code.

- **Purpose**: Allows UI structure to be defined in a *declarative* manner directly in JavaScript, rather than using strings or DOM APIs. This improves readability and creates a closer connection between markup and logic.

- **Under the Hood**: JSX is transpiled by the build process (Babel) into regular JavaScript code (e.g., React.createElement calls). However, the developer writes code that closely resembles HTML markup, which makes it easier to get started.

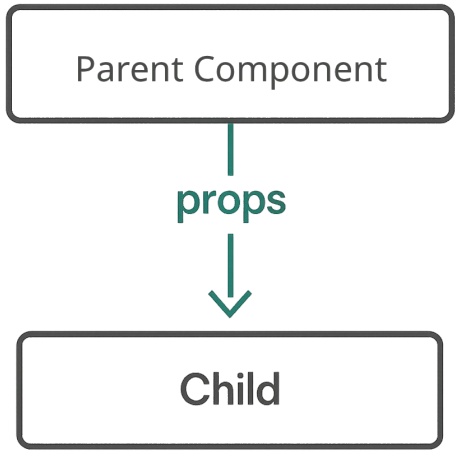
- In JSX, **curly braces { }** are used to embed JavaScript expressions

```
const myElement = <h1>React is {5 + 5} times better with JSX</h1>;
```



Props (Properties)

- **Information flow** from parent to child nodes
- **Props** are input parameters of a component. They are passed from the **parent element** to the component, similar to function arguments.
- Props are **read-only**: A component should not modify its props—they are used solely for rendering or configurable behavior.
- Examples of props: text, numbers, callback functions, or even complex objects. For example: `<Button label="Save" onClick={...} />` – `label` and `onClick` are props of the button.
- Callback functions used as props also enable information flow from child to parent nodes





Props: Example

```jsx
import PersonListItem from "./PersonListItem";

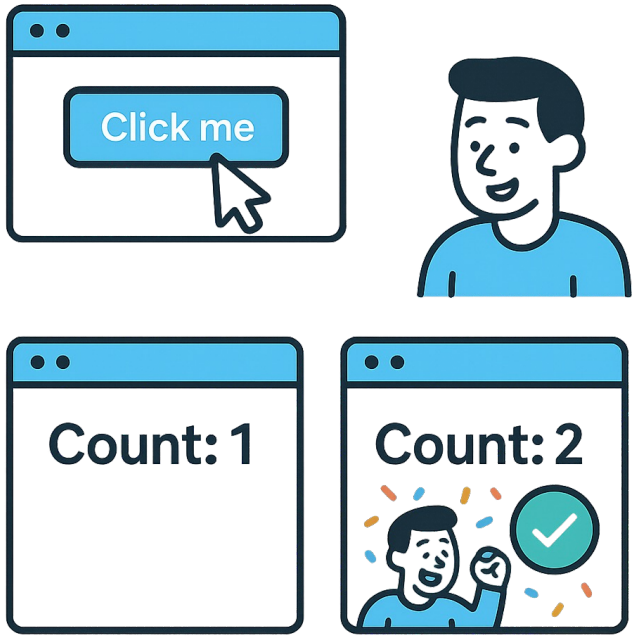
export default function PersonsList({ persons }) {
  return (
    <table>
      <thead>
      <tr>
        <th>ID</th>
        <th>First Name</th>
        <th>Last Name</th>
        <th>Position</th>
     </tr>
     </thead>
     <tbody>
       {persons.map((person) => (
         <PersonListItem key={person.id} person={person} />
       ))}
     </tbody>
   </table>
);
}
```

```jsx
export default function PersonListItem({ person }) {
  return (
    <tr>
      <td>{person.id}</td>
      <td>{person.firstName}</td>
      <td>{person.lastName}</td>
      <td>{person.position}</td>
    </tr>
  );
}
```



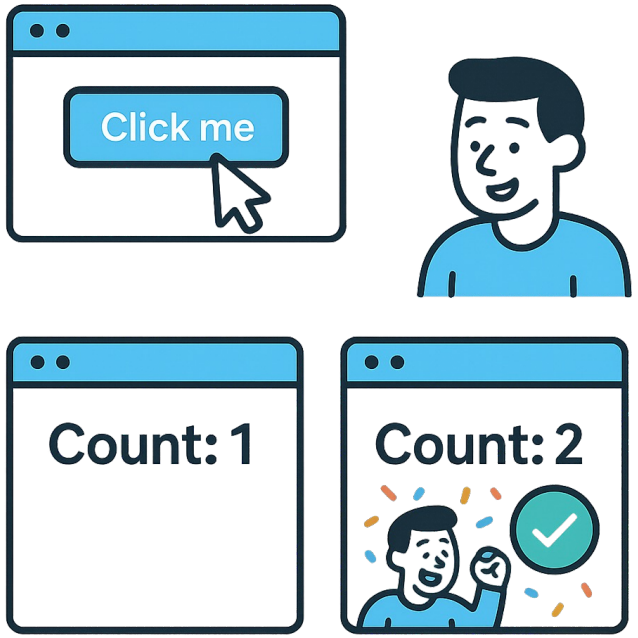
React – Client-Side State

- **State** refers to a component's internal, mutable data. Unlike props, which come from outside, state *is managed within* the component.
- State enables **dynamic UIs**: Changes to state trigger a re-render of the component (and its children), ensuring that the UI always reflects the current state.
- In functional components, state is managed via React **hooks** , e.g., `const [count, setCount] = useState(0);` defines a state value `count` with an initial value of 0 and an update function.



React – client-side state

- **Multiple state values** per component are possible (multiple useState calls or complex state objects). However, you should keep state to a minimum—only store what is necessary for rendering and interaction.
- **Lifecycle**: Similar to Angular lifecycle hooks (ngOnInit, etc.), React provides mechanisms to react to state and prop changes (e.g., `useEffect` hook).



State: Example

```
import { useEffect, useState } from "react";
import PersonsList from "./components/PersonsList";

export default function Home() {
  const [persons, setPersons] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    const fetchPersons = async () => {
    try {
      const response = await fetch("/api/persons");
      if (!response.ok) {
        throw new Error("Failed to load persons.");
      }
      const data = await response.json();
      setPersons(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };
  fetchPersons();
}, []);
```

```
…
return (
  <main>
  <h1>People</h1>


  {loading && <p>Loading people...</p>}

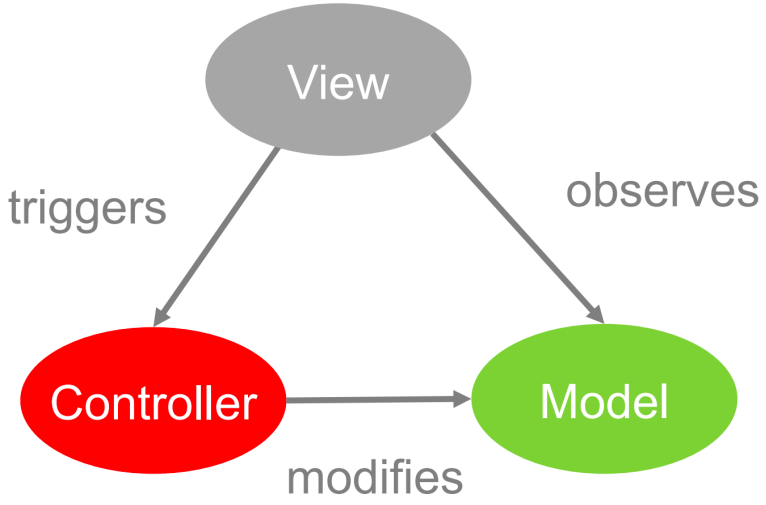

  {error && !loading && (
     <p> {error} </p>
   )}


  {!loading && !error && (
     <div>
       <PersonsList persons={persons} />
     </div>
  )}
  </main>
);
}
```



React: MVC?

- **React ≠ a classic MVC framework**: It primarily provides the view layer. There is no prescribed controller or model—that responsibility lies with the surrounding structures of the app.
- People often say that React is **“the V in MVC”**. React components represent views and contain some logic, but there is no strictly separate controller layer.
- A React component tree can nevertheless be part of an MVC-like pattern: for example, model data comes from external sources (API, state), the component renders it (view), and handles user events (controller logic in event handlers).





Architectural Classification of React

- **Component-Based Paradigm**: React follows its own architectural style—components manage their state (Model) and render the UI (View) in a single unit, sometimes also handling events (Controller).
- This results in **high cohesion**: Each component is responsible for a clearly defined part of the UI.
- For large-scale applications, **structuring patterns** are implemented: e.g., Flux/Redux as a unidirectional data flow pattern (state as a central *store* model, actions as controller logic, React components as the view).
- It's important to note that React gives developers freedom. It doesn't enforce any specific pattern. You can integrate React into MVC-like layers (e.g., React for the view and controller, external services for the model) or overlay it with custom architectures.
- **Conclusion**: React itself is lean and flexible—it's up to the developer to establish architectural principles. It is intentionally "just" the view library, but this also means great freedom and adaptability to various architectural patterns.



HOCHSCHULE
HANNOVER
UNIVERSITY OF
APPLIED SCIENCES
AND ARTS
–
Fakultät IV
Wirtschaft und
Informatik

Fortgeschrittene Aspekte verteilter Systeme

Next.js



Setting Up a New Next.js Project

- **Create a project**: Run the following command in the terminal:
  `npx create-next-app@latest my-react-app -js`
  (uses defaults; alternatively, run without flags, and the CLI wizard will prompt you for details).

- CLI prompt: Enter project name, TypeScript = Yes, ESLint = Yes, CSS framework (e.g., Tailwind) optional, use src/ directory = Yes, App Router (recommended) = Yes. *(These options may vary slightly depending on the Next.js version.)*

- After confirmation, the CLI creates all files and installs dependencies (React, Next, Webpack, etc.).

- **Switch to the project**: `cd my-react-app`.
  Then start the development server with `npm run dev`.
  The terminal output displays the local URL (default: `http://localhost:3000`).

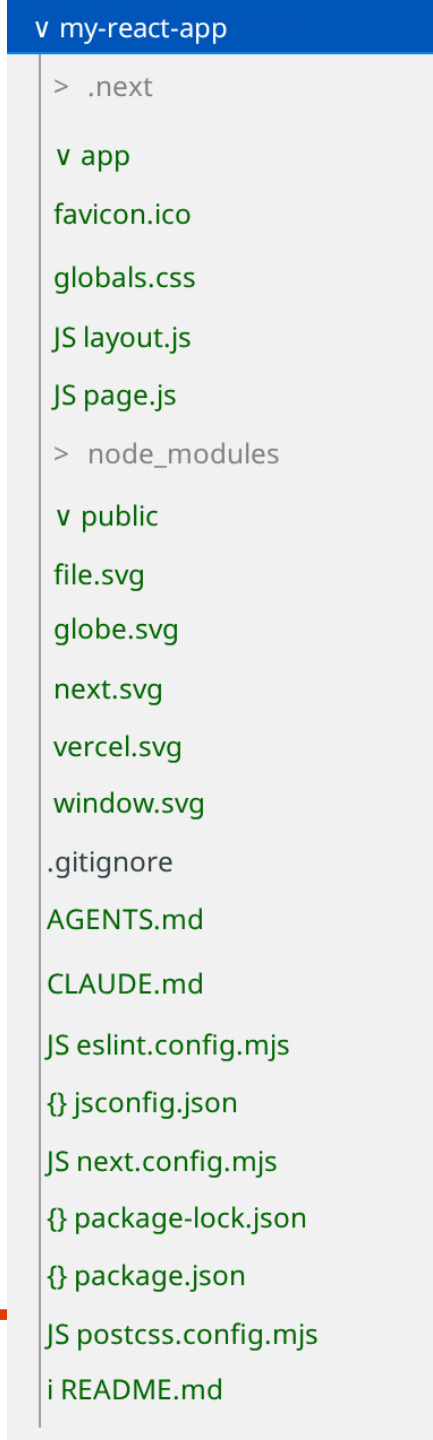
- **First test**: Open `localhost:3000` in your browser—the default `Next.js` welcome page should appear (confirming that setup was successful).



Project Structure (Next.js)

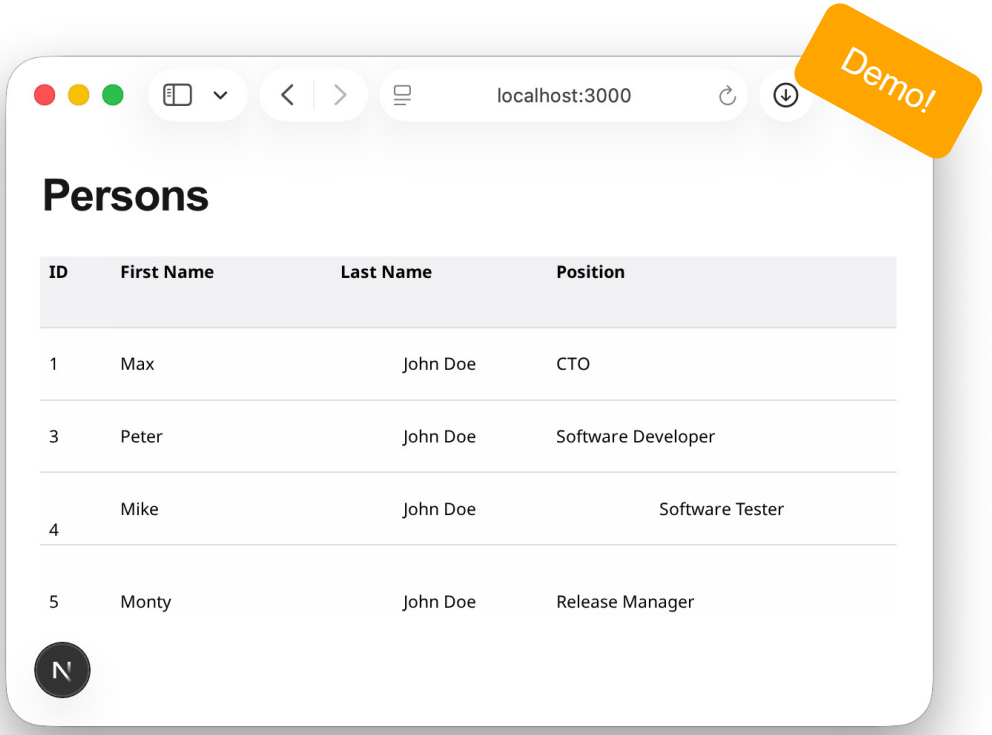
- After generation, you'll see the following in the project: *(VS Code Explorer screenshot with main folders/files highlighted)*
  - `app/` (or `pages/` for legacy routers): contains page components. For example, `app/page.js` (home page component), `layout.js` (root layout; must have `html` tag), global CSS styles (`globals.css`)
  - `public/`: Static files (e.g., `favicon.ico`, possibly a directory for images).
  - `package.json`: List of dependencies (including *React*, *react-dom*, *next*) and scripts (dev, build, start, etc.).
  - Additional configurations: `jsconfig.json` (JS settings), `.eslintrc.json` (linting rules), etc., which have been predefined by the CLI.

- **Special feature**: Unlike Angular, there is no **module structure** — Next.js is organized primarily by pages. Components can be freely organized into folders, depending on the developer's preferences.



Display Person Data

- We've already looked at the code for the main components (PersonList, PersonListItem, Home).



Styling with Tailwind

- Tailwind is a CSS framework designed to simplify consistent styling of the application
- With Tailwind, components are styled by writing predefined classes (utility classes) directly into the markup.

- `mx-auto`: centers the page container horizontally.
- `flex`: makes the main element a flex container.
- `w-full`: uses the full available width.
- `max-w-4xl`: limits the width so that the content does not become too wide.
- `flex-1`: allows this area to expand to fill the remaining vertical space in the parent layout.
- `flex-col`: stacks child elements vertically.
- `p-6`: base padding on all sides.
- `md:p-10`: larger padding on medium+ screens.

```
<main className="mx-auto flex w-full max-w-4xl flex-1 flex-col p-6 md:p-10">
```



Styling with Tailwind

- Contradicts many traditional best practices (similarity to inline styles)
- Possible advantages:
  - Speed: no need to define class names or selectors yourself
  - Impact of changes is limited to the local scope
  - Supports a component-based approach
  - CSS does not continue to grow
- More info: https://tailwindcss.com/

- Tailwind is not required; other CSS frameworks or even completely custom styles can be used.



Routing in Next.js: File-system-based approach

- A key advantage of Next.js is its built-in **routing system (App Routing)**, which is based on the file system. Each directory in the root folder automatically corresponds to a route (URL) in the application—no additional router library is required.
- Next.js identifies pages based on the folder name: for example, `/page.js` results in the home page `/`, a file `aboutus/page.js` results in the route `/aboutus`.
- Routing in Next.js is therefore **very simple and convention-based**: Developers simply create or name folders to define new pages and routes without having to manually configure routes.
- In summary: Next.js drastically simplifies routing because **folder name = URLs** .

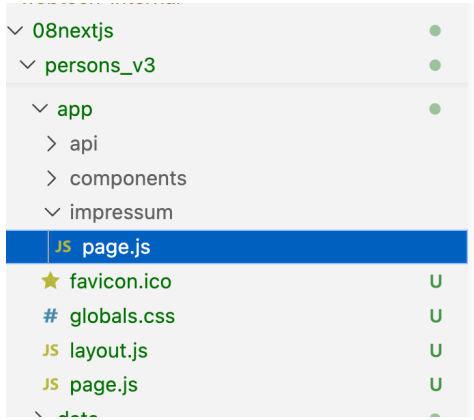


Static Routes in Next.js

Static (fixed-name) pages can be implemented in Next.js simply by creating folders and files in the `app` directory.

making a page automatically available at the URL

In the example: `impressum/page.js` results in a route to `/impressum`




```
export default function Impressum() {

  return (
    <main className="mx-auto flex w-full max-w-4xl flex-1 flex-col p-6 md:p-10">
      <h1 className="mb-6 text-3xl font-bold tracking-tight">Impressum</h1>
      <p>
        Max Mustermann <br />
        Musterstraße 1 <br />
        12345 Musterstadt <br />
        Germany
      </p>
    </main>
  );
}
```



Navigation between pages

For links, Next.js provides the `<Link>`component (imported from `'next/link'`).

`<Link>` component ensures that the new page loads in the background and is seamlessly switched to (for a smoother user experience)

`<a>` tag with an `href` attribute is also possible, but it causes the page to reload completely

```
<p>
  <Link href="/">
  Back to the home page (without reloading)
  </Link>
</p>

<p>
  <a href="/">
  Back to the home page (with reload)
  </a>
</p>
```



Dynamic Routes

In addition to fixed routes, Next.js also supports **dynamic routes**, i.e., pages with variable URL parameters. This is achieved using **square brackets** in the filename.

A file named `/api/persons/[id]/route.js` defines a dynamic route, for example. When a GET request is made to `/api/persons/3`, Next.js then executes the corresponding function in route.js.

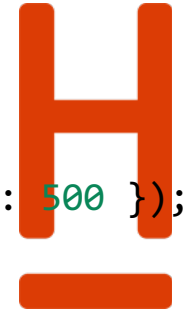
The name inside the brackets (here `[id]`) is made available in the function as the **parameter.**

```
export async function GET(_request, { params }) {
  try {
    const { id: rawId } = await params;
    const id = Number(rawId);

    if (!Number.isInteger(id)) {
       return Response.json({ error: "Invalid id." },
                  { status: 400 });
    }

    const persons = await loadPersons();
    const person = persons.find(
                (person) => Number(person.id) === id);

    if (!person) {
      return Response.json(
          { error: "Person not found." }, {status: 404
      });
    }
    return Response.json(person);
  } catch {
    return Response.json(
       { error: "Could not read data." }, { status: 500 });
  }
}
```



Pre-Rendering in Next.js

One of the most powerful features of Next.js is **pre-rendering of pages** (pre-rendering). Unlike traditional SPAs (purely client-side), Next.js pages can already be **fully rendered HTML** (cf. PHP). There are two main strategies for this:

- **Server-Side Rendering (SSR):** The page (not the entire website) is **rendered on the server for every request** (HTML is generated on the fly).

- **Static Site Generation (SSG):** The page (not the entire website) is **generated once in advance** (e.g., during the build) as a static HTML file and then reused for subsequent requests.

Next.js supports both approaches (and even a hybrid).
You can specify which strategy to use on a per-page basis.
Through SSR and SSG, Next.js can combine the advantages of classic multi-page server apps with the benefits of React.
In traditional React SPAs, the browser initially sees only a blank page and then builds the UI using JavaScript—Next.js circumvents this by already **pre-rendered HTML**, which leads to faster page loading and better SEO.



Server-Side Rendering (SSR)

With SSR, the server generates the full HTML page anew for **each user request** (possibly after database/API queries) and sends it to the client.

The browser thus receives ready-made HTML that can be displayed immediately, while React “hydrates” in the background—that is, it brings the page delivered by the server to life and makes it interactive.

For the user, this results in a fast initial page load with up-to-date data, and search engines see complete content in the HTML.

SSR is particularly well-suited for content that is **highly dynamic or personalized** and must be up-to-date with every request.

However, the server must process every request, which slightly increases the **time-to-first-byte** — the page takes a little longer to load from the server.



SSR in Next.js

In Next.js, you don’t need to explicitly enable SSR, since every page and component is SSR by default, meaning they are rendered on the server.

The person data can also be loaded using SSR. In this case, the API call is replaced by a local function call on the server.

```
import { loadNormalizedPersons } from "../lib/persons-store";
import PersonsList from "./components/PersonsList";

export default async function SsrPersonsPage() {
  let persons = [];

  try {
    persons = await loadNormalizedPersons();
  } catch {
    error = "Could not load persons data.";
  }
  return (
    <main>
         <h1>Persons</h1>
         <div>
             <PersonsList persons={persons} />
         </div>
   </main>
);
}
```



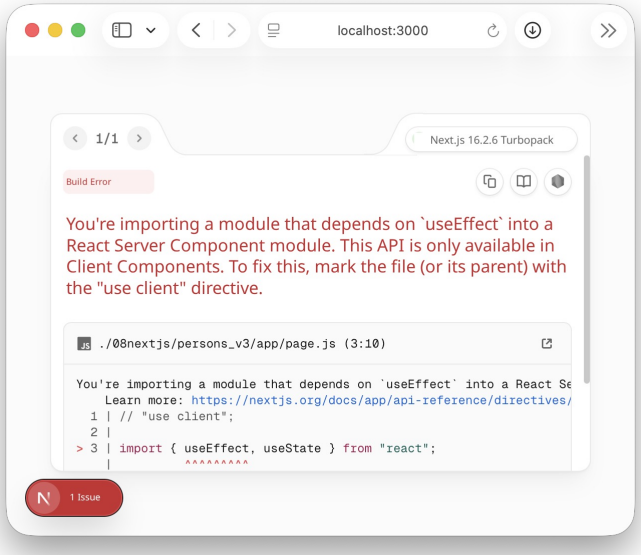
CSR in Next.js

If a page or component is to be explicitly rendered by the client, you must use “`use client`.”

Some components cannot be rendered on the server, e.g., if a React state is stored (on the client side).

Error message if “`use client`” is omitted.

SSR is also performed with CSR, but the page is then dynamically “hydrated” on the client.



page.js U

webtech-internal > 08nextjs > persons_v3 > app > page.js > ...

```
1  "use client";
2
3  import { useEffect, useState } from "react";
4  import PersonsList from "./components/PersonsList";
5
6  export default function Home() {
7    const [persons, setPersons] = useState([]);
8    const [loading, setLoading] = useState(true);
9    const [error, setError] = useState("");
10
11   useEffect(() => {
12     const fetchPersons = async () => {
13       try {
14         const response = await fetch("/api/persons");
15
16         if (!response.ok) {
17           throw new Error("Failed to load persons.");
18         }
19
20         const data = await response.json();
21         setPersons(data);
22       } catch (err) {
23         setError(err instanceof Error ? err.message :
```

Outlook: Static Site Generation (SSG)

With SSG, pages **are generated** (build time) as static HTML files. These pre-generated files are stored on the server (or CDN) and can be delivered immediately in response to user requests, **without** the server having to render the page. The result: extremely fast delivery and low server load, since the content is already fully prepared.

SSG is ideal for pages whose content **rarely changes** or is updated upon deployment (e.g., a blog, documentation pages, marketing pages). SEO is excellent because complete HTML is available, and load times are optimal—the server can send the file immediately, and it may even be cached worldwide via a content delivery network (CDN caching).

The downside is obvious: When content changes, **rebuilt** (re-deployed) so that the HTML file is updated. Without further measures, users will otherwise see outdated content. Therefore, SSG is ideal for relatively static content but less suitable for data that is updated very frequently.



Summary

- We have used React as **component-based** UI library that emerged from the need to better handle complex UIs.
- **Next.js** extends React with key features (routing, SSR, …), allowing us to quickly build working projects.



HOCHSCHULE
HANNOVER
UNIVERSITY OF
APPLIED SCIENCES
AND ARTS
–
Fakultät IV
Wirtschaft und
Informatik

Fortgeschrittene Aspekte verteilter Systeme

Spring Framework + Spring Boot



What is the Spring Framework?



What is the Spring Framework?

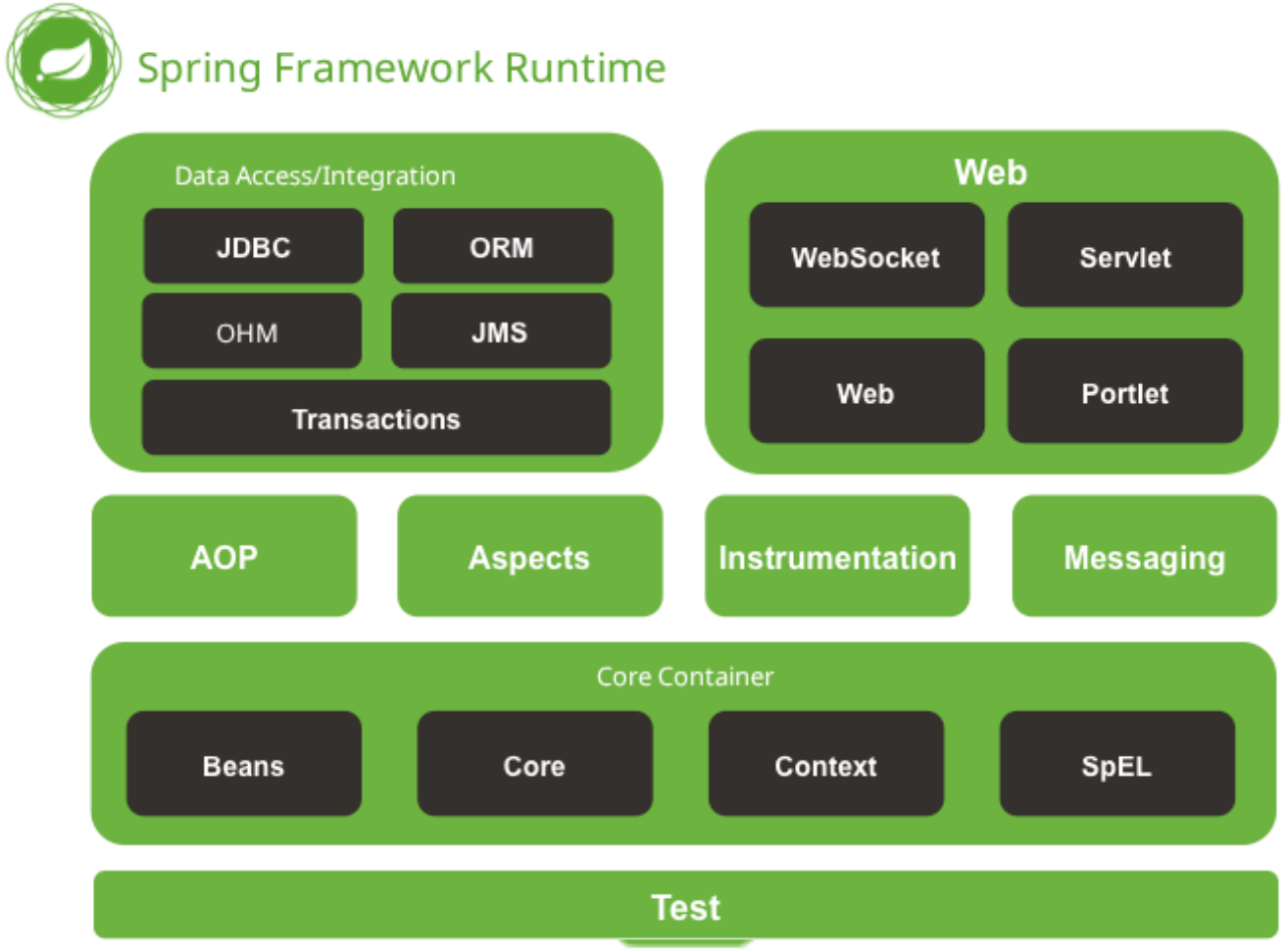
- Application framework for the Java platform
- Developed by VMware and Pivotal Software Inc.
- Commonly used as *a supplement or replacement for Enterprise Java Beans (EJB)*
- Open source at:
  - Spring Framework: https://github.com/spring-projects/spring-framework
  - Other Spring projects: https://github.com/spring-projects

Main Goals:

- Simplify the development of Java Enterprise applications
- Promote good programming practices



Core Architecture / Modules of Spring



Core Architecture / Modules of Spring

`Spring Core and Spring Beans`

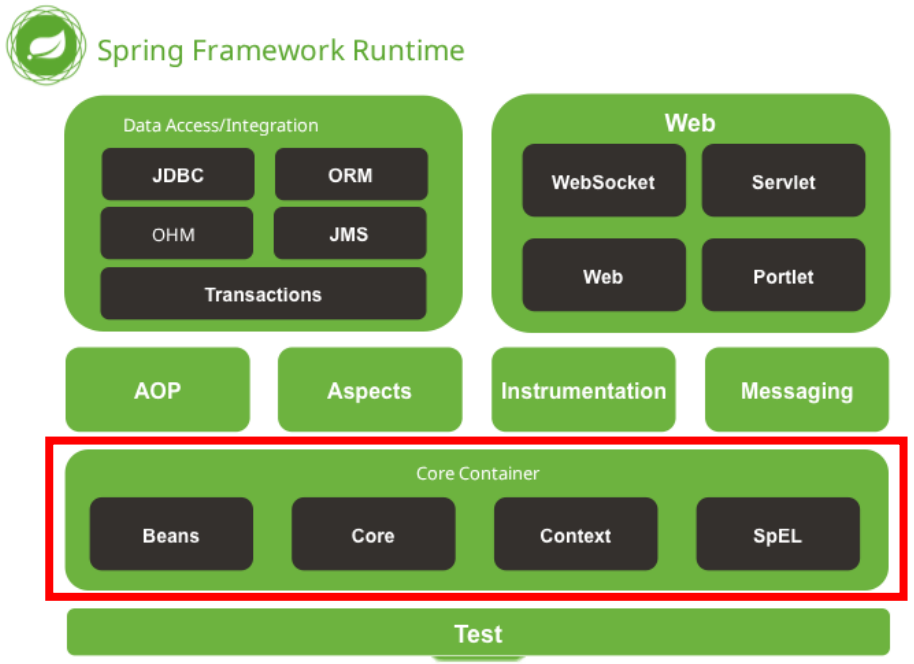
- Provide the framework's basic functionality, including Inversion of Control (IoC) and Dependency Injection (DI).
- `with` BeanFactory as an implementation of the Factory pattern.

`Spring Context`

- Builds on `spring-core` and `spring-beans` .
- Provides functionality for accessing objects in a "framework-style" manner (similar to a JNDI registry).
- Also supports Java EE features, such as EJB or JMX.

`Spring SpEL`

- Provides an expression language for querying and manipulating object graphs at runtime.



Core Architecture / Modules of Spring

`Spring AOP`

- Enables interceptors (“An interceptor extends a framework or middleware without modifying it itself.”)

`Spring Aspects`

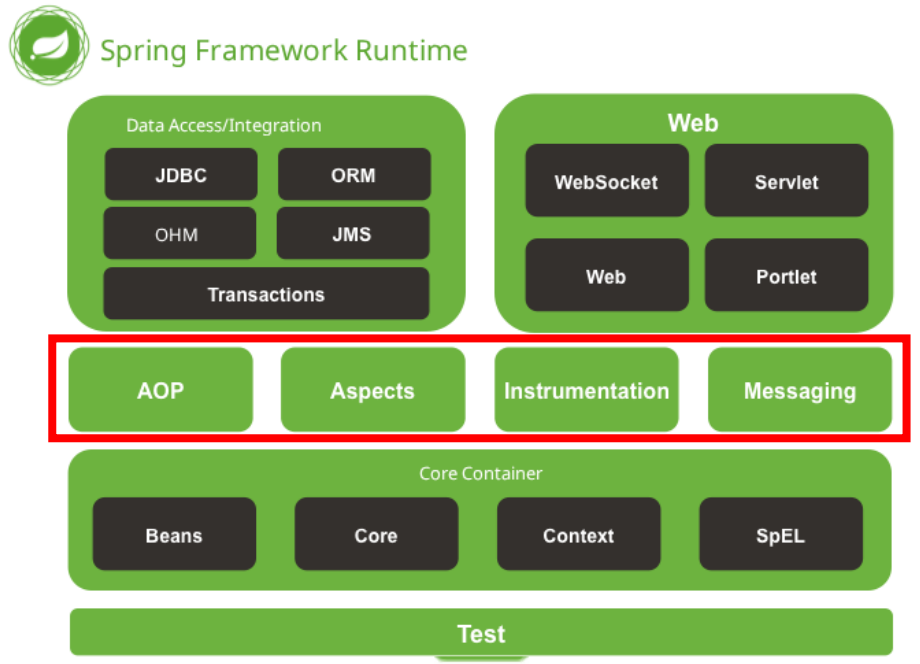
- Enables integration with AspectJ.

`Spring Instrument`

- Allows Java classes to be instrumented (modifying bytecode at runtime).
- Adds ClassLoader implementations for various application servers (e.g., spring-instrument-tomcat)

`Spring Messaging`

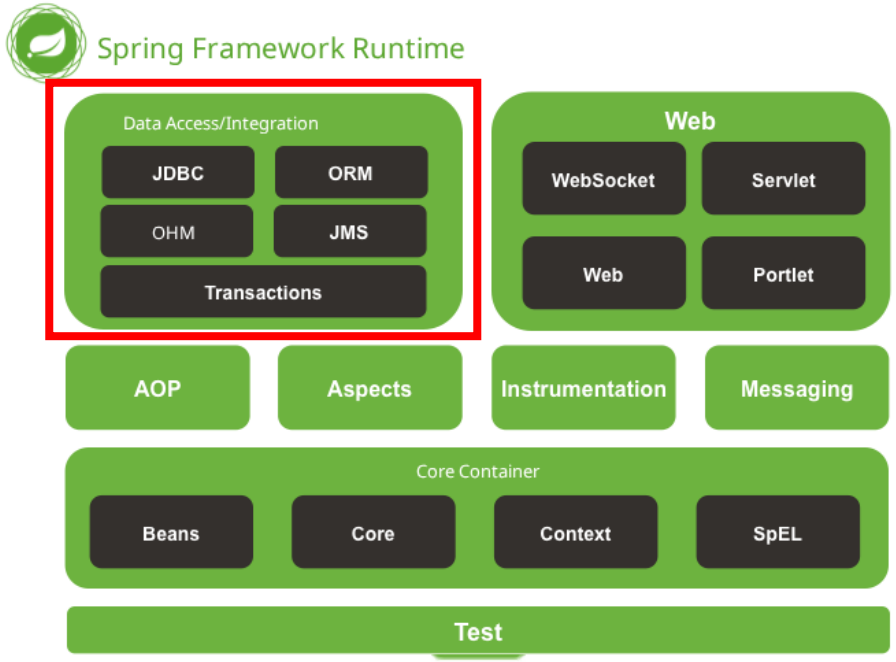
- Support for STOMP (Simple Text Oriented Messaging Protocol)



Core Architecture / Modules of Spring

`Data Access/Integration`

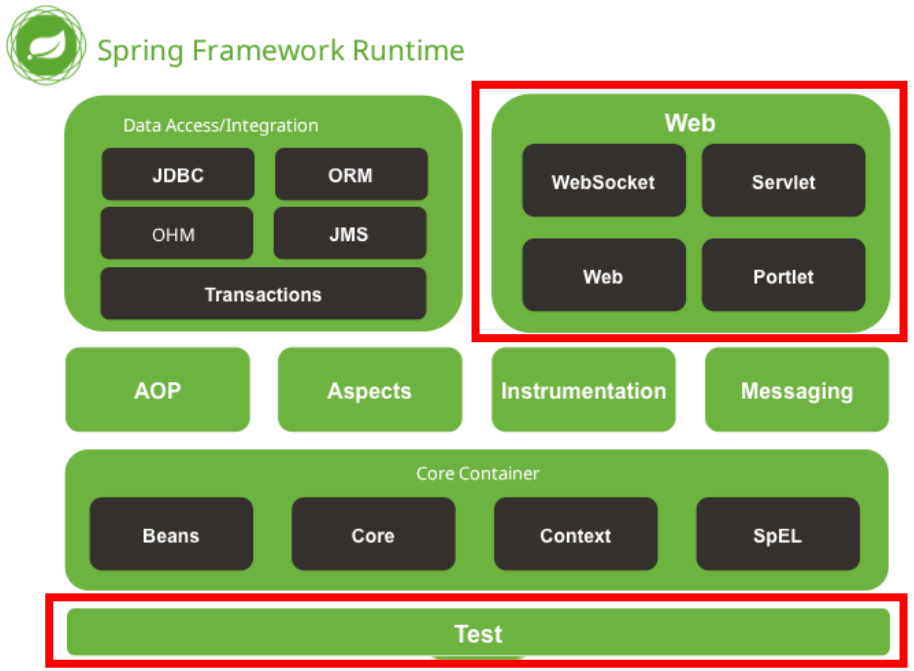
- **`Spring JDBC`** abstraction layer
- **`Spring ORM`** provides integration layers for common object-relational mapping APIs, including JPA, JDO, Hibernate, and iBatis.
- **`Spring OXM`:** an abstraction layer for object-to-XML mapping implementations for JAXB, Castor, XMLBeans, JiBX, and Xstream.
- **`Spring JMS`** (Java Messaging Service) provides functionality for sending and receiving messages.
- **`Spring Transactions`** for programmatic and declarative transaction management for classes that implement specific interfaces.



Core Architecture / Modules of Spring

`Web`

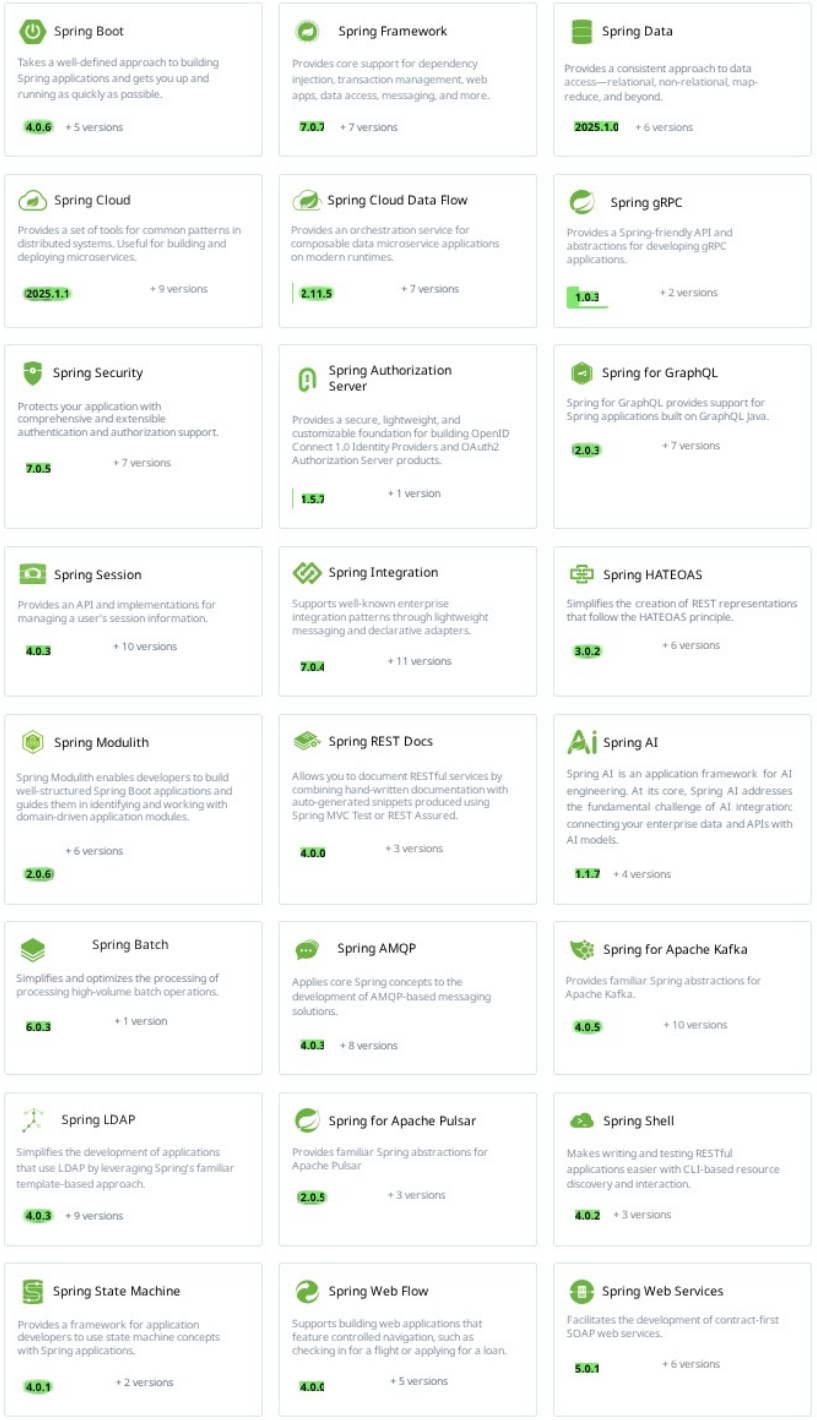
- **`Spring Web`**: Basic functionalities for creating web applications
- **`Spring Web MVC (Servlet)`**: Implementation of the Model-View-Controller pattern and support for creating REST web services.
- **`Spring WebSocket`**: WebSocket-based bidirectional communication between client and server in web applications.
- **`Spring Portlet`**: Provides the MVC implementation for use in a portlet environment and mirrors the functionality of the Web Servlet module.



`Test`: Supports component and integration testing of Spring modules using JUnit or TestNG.

Spring Projects

- Different Spring projects have been created to address many specific issues; see https://spring.io/projects



Spring Boot



Spring Boot

- "…makes it easy to create stand-alone, production-grade Spring-based applications that you can 'just run'.
- We take a strong stance of the Spring platform and third-party libraries so you can get started with minimal hassle. Most Spring Boot applications require minimal Spring configuration."

Open Source:

- https://github.com/spring-projects/spring-boot



Phil Webb is a contributor to Spring
https://github.com/philwebb

Spring Boot



- Spring Boot offers a "convention-over-configuration“ solution for creating production-ready, standalone Spring-based applications
  →“(…) Applications that you can ‘just run’.”
- *a popular choice for developing* ***microservices***

- Advantages:
  - Spring applications that are easy to understand and develop
  - Increases productivity
  - Reduces development time



Spring Boot and Build Systems

Spring Boot uses build systems

- Goal: To automate as many steps of the build process as possible (compiling, testing, “packaging,” and distributing the software)
- Examples:
  - Apache Maven
  - Gradle

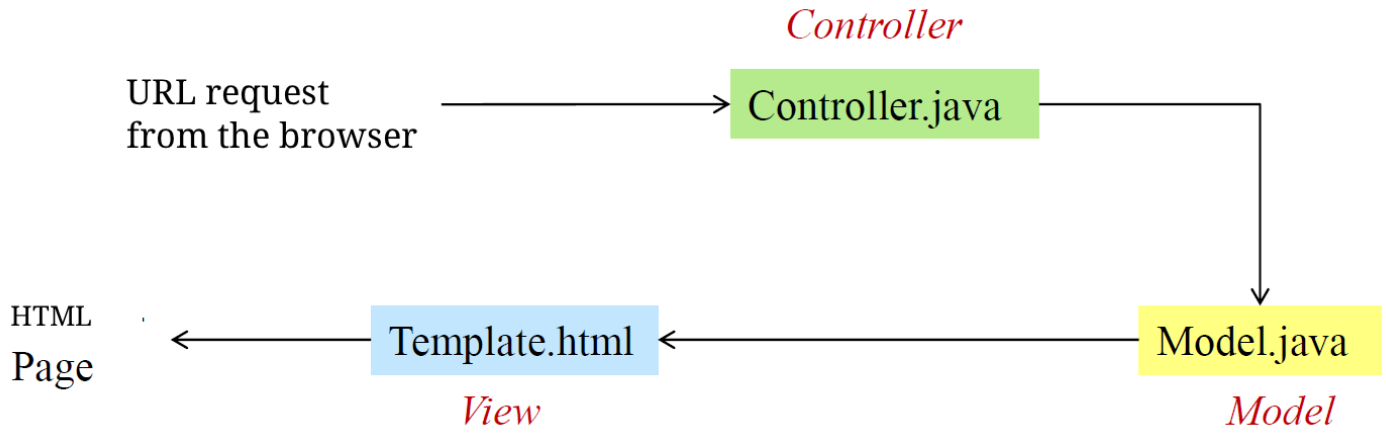




https://de.wikipedia.org/wiki/Gradle#/media/Datei:Gradle_logo.png

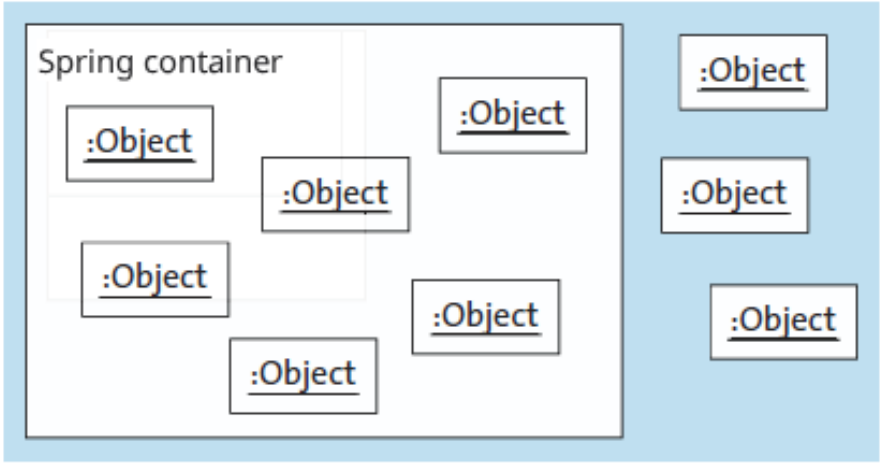
Spring MVC

- Is a component of the Spring Framework for building web applications
- It follows the MVC architecture (**Model-View-Controller)**
- Structures the application into:
  - Model: Data models (Java objects)
  - View: Template that generates the HTML page
  - Controller: responds to HTTP requests from the browser



Spring-Managed Beans Container

- When a Spring application starts, a container for beans is initialized
- Beans ~ Components
- Container
  - Manages beans (add, query, delete)
  - Automatically configures dependencies (references) between beans
- Adding beans:
  - Declaratively (via annotations)
  - or manually register
- References between beans via injection

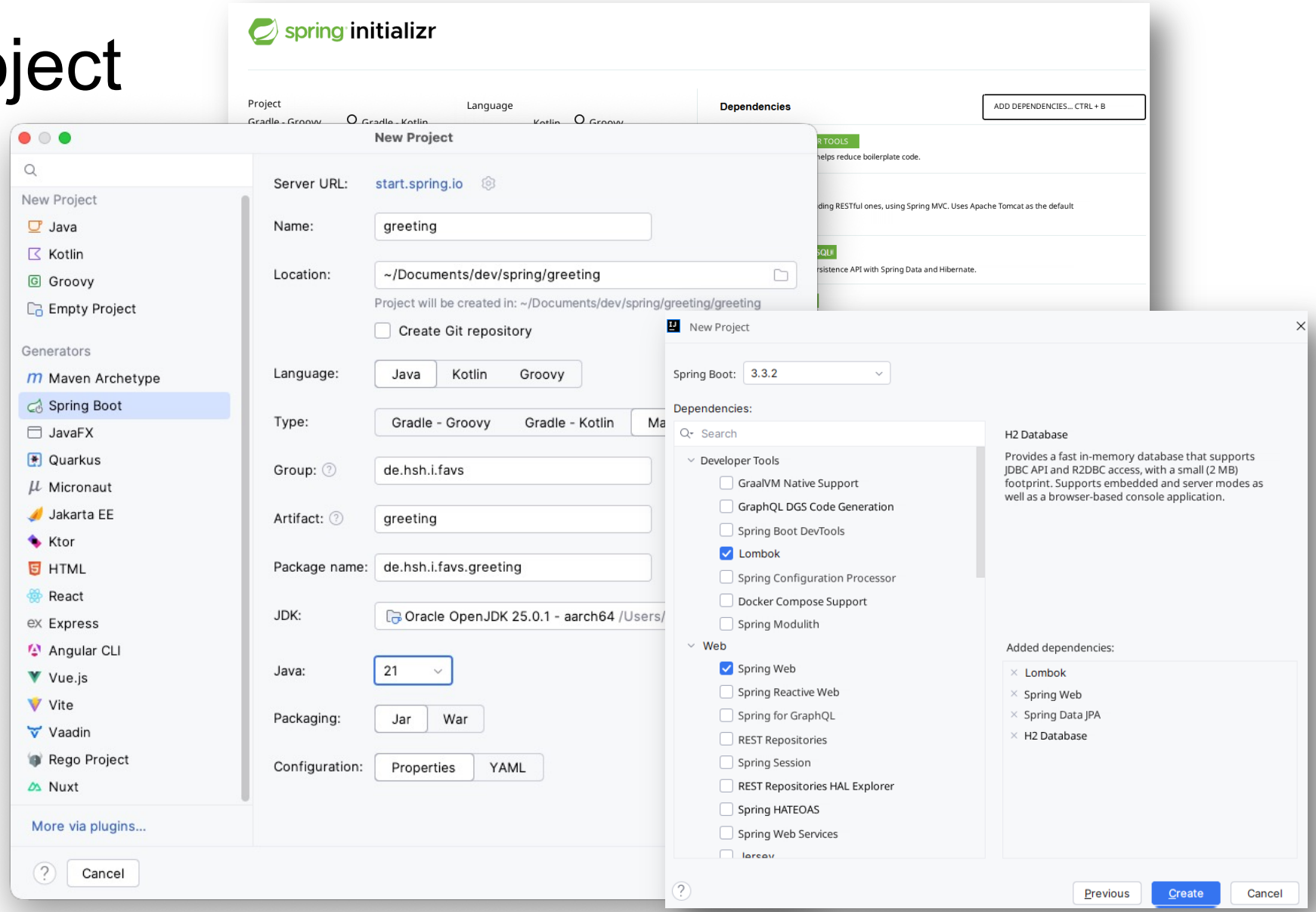


Source: Spring Boot 3 and Spring Framework 6, Christian Ullenboom, 2023

@Component	– any component
@Repository	– for data storage
@Service	– for application logic
@Controller	– requests from the front end

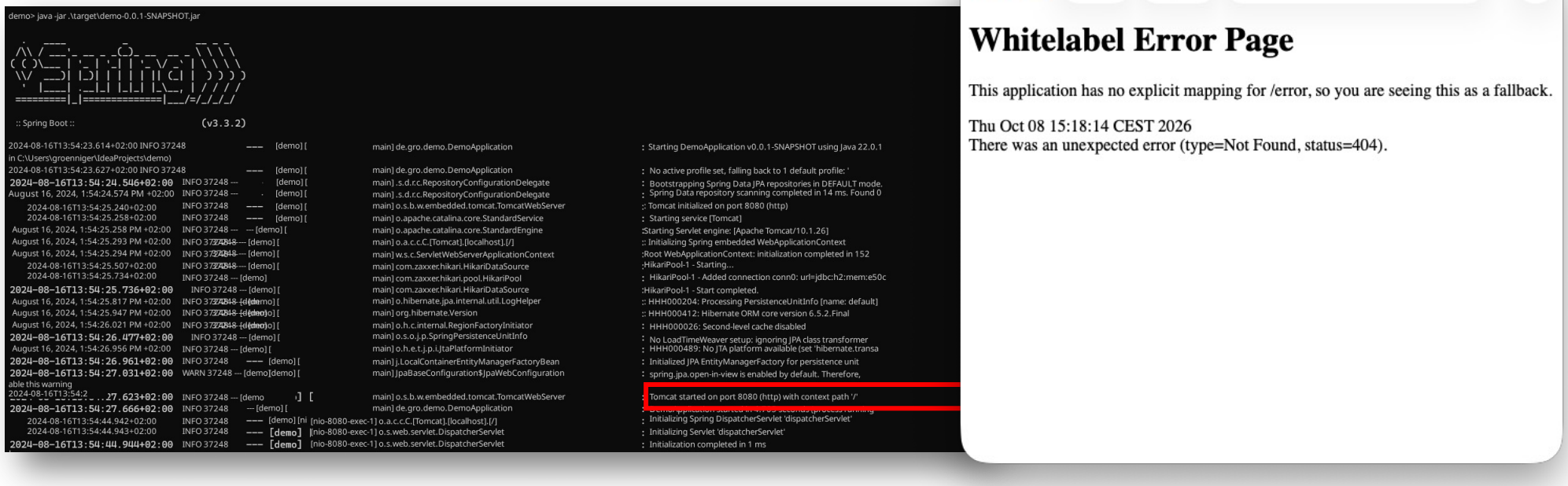
Spring Boot Project Setup

- Using `Spring Initializr`
- Either in the IDE (here, IntelliJ) or via the web at start.spring.io
- Initial list of dependencies on libraries such as web, databases, etc.
- Add the Lombok annotation framework



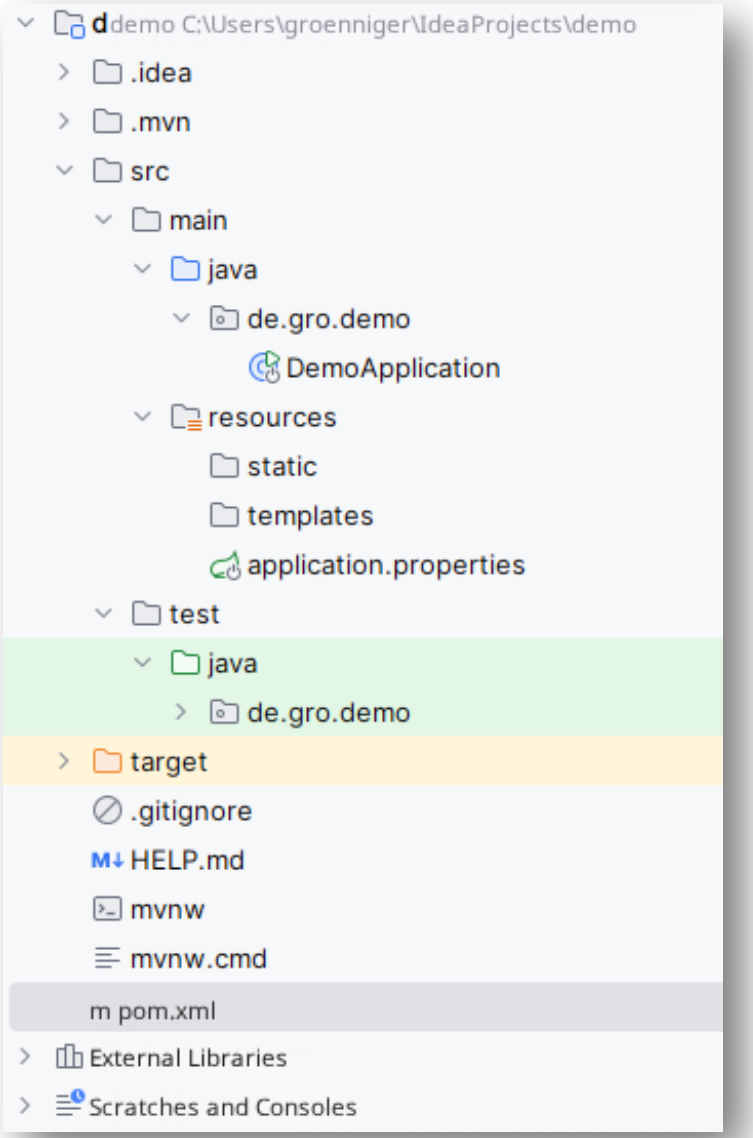
Setting up a Spring Boot project

- The generated project is ready to run immediately
- Via an appropriate Maven target (`package`), a JAR file is generated
- If a dependency on Spring Web is set, an embedded web server starts automatically



Spring Boot project

- Directory Structure and Files
  - `pom.xml`
    - Project Object Model, Maven configuration
    - Definition of dependencies
    - There is also a parent POM
  - `application.properties`
    - Spring Application Configuration
    - actually for the default profile
    - In the folder `resources` , additional non-source-code artifacts
  - Java packages containing source code for the application and tests
  - `HELP.md` provides links to documentation depending on the configured dependencies



Spring MVC

- **Maven**

  `pom.xml` (an excerpt)

```
<?xml version="1.0" encoding="UTF-8"?>
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.0.6</version>
        <relativePath/> <!-- Look up parent from repository -->
    </parent>
…
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-mustache</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <scope>annotationProcessor</scope>
        </dependency>
    </dependencies>
…
```



Spring MVC

GreetingController.java

- **Controller**

```
@Controller
public class GreetingController {

    @GetMapping("/greeting")
    public String greeting(Model model) {
        model.addAttribute("name", "World");
        return "greeting_template";
    }

}
```

URL for the request

Extend the model

Returns the template name used to generate the HTML page.



Spring MVC

`greeting_template.html`

- **View**

```
<html>
  <body>
    <p>Hello, {{name}}</p>
  </body>
</html>
```

The templates specify placeholders with attribute names from the model.
When the HTML page is generated, these are replaced with the corresponding attribute values from the model.
Here is an example using the attribute `name`.

- By default, Mustache uses the `.mustache` file extension for template files, so in the `application.properties` or `application.yaml` add the following for HTML:

`application.properties`

```
spring.mustache.suffix:.html
```

`application.yaml`

```
spring:
  mustache:
    suffix: .html
```



Spring MVC

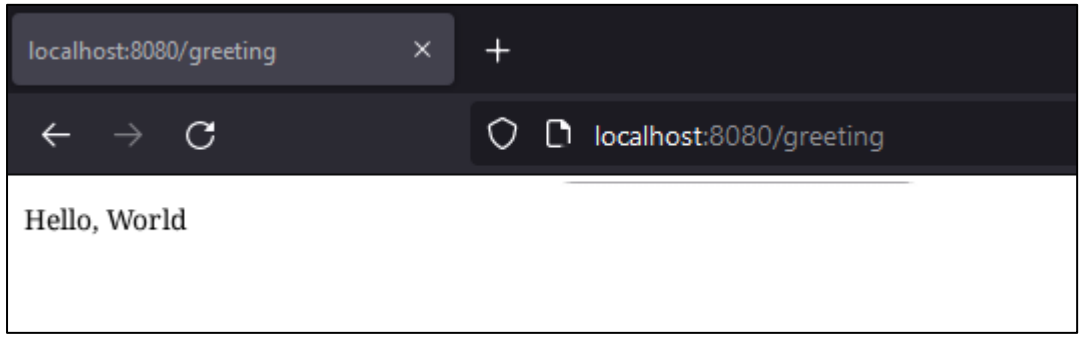
SpringMvCdemoApplication.java

- **Main Application**

```java
@SpringBootApplication
public class SpringMvCdemoApplication {

   public static void main(String[] args) {
      SpringApplication.run(SpringMvCdemoApplication.class, args);
   }

}
```



REST with Spring Boot

Spring MVC for REST:

Allows the application to be structured into:

- Model:
  - Data models (Java objects)
- View:
  - Application (Client): Generates the view using templates
  - REST: Converts the model to JSON
- Controller:
  - Controller that responds to HTTP requests made by the client

- The H2 in-memory database will be used for storage: For this, we use Spring Boot Data JPA as a dependency



REST with Spring Boot

- **Model + Standard Repository**

```
@Entity
@Getter
@Setter
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private Double price;
    private String author;

}
```

Annotation as a persistence object; requires an ID attribute

Lombok annotation

```
public interface BookRepository extends JpaRepository<Book, Long> {
}
```

Automatic persistence layer with methods such as save, delete, findBy, etc.



REST with Spring Boot

- **Controller**

```
@RestController
public class BookController {

    @Autowired
    private BookRepository bookRepository;

    @GetMapping("/books")
    public List<Book> getBooks() {
        return bookRepository.findAll();
    }

    @GetMapping("/books/{id}")
    public Book getBookById(@PathVariable Long id) {
        return bookRepository.findById(id).get();
    }
```

The controller class is annotated with `@RestController`.

Injecting the repository (preferably as a constructor argument these days)

A list of books is returned as a JSON object.

A resource's ID is usually included in the URL. To access it, we use `@PathVariable`.

REST with Spring Boot

- **Controller**

```
@RestController
public class BookController {

    @Autowired
    private BookRepository bookRepository;

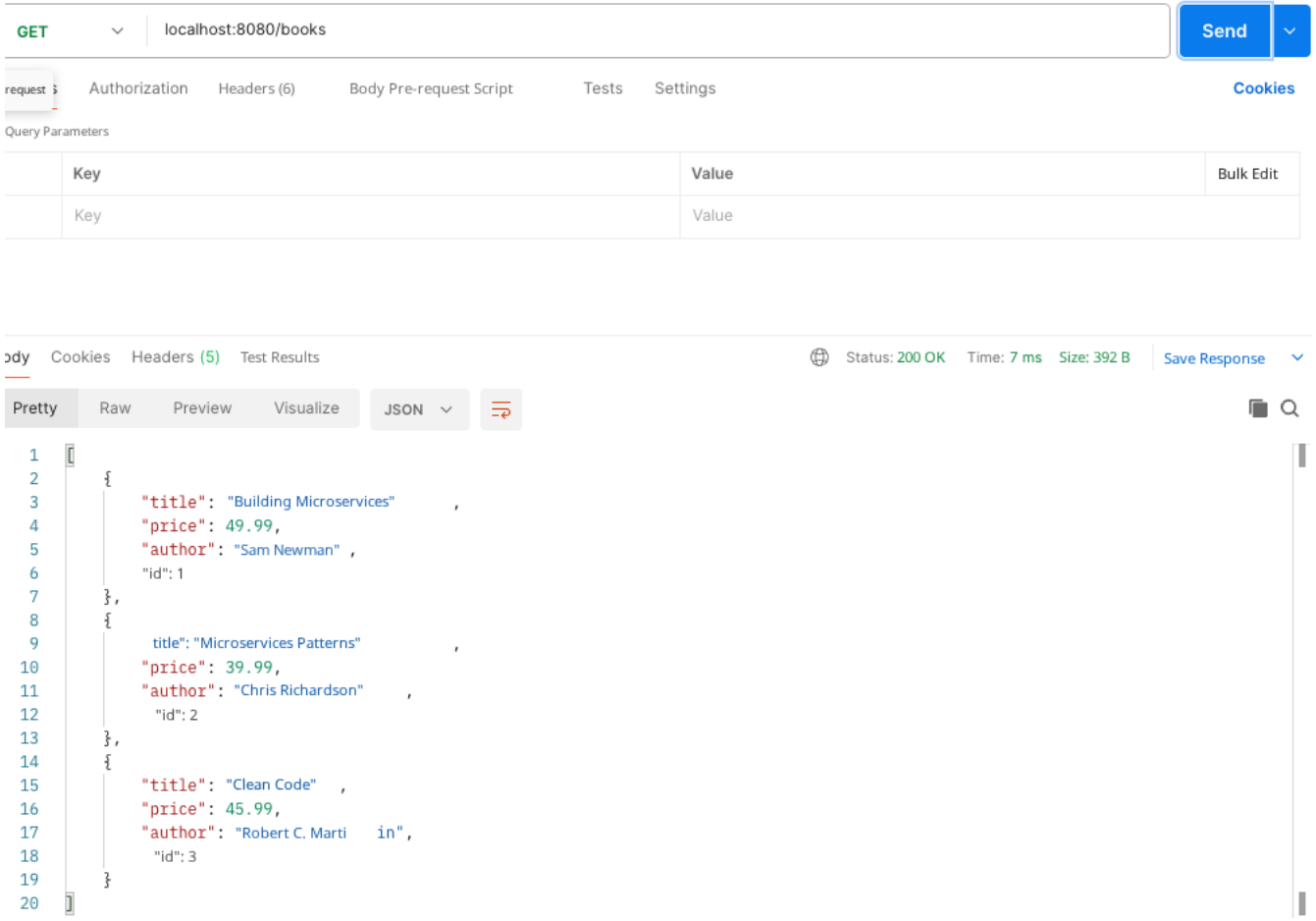
    @GetMapping("/books")
    public List<Book> getBooks(@RequestParam
                               Optional<String> author) {
        if (author.isPresent()) {
            return bookRepository.findByAuthor(author.get());
        }
        return bookRepository.findAll();
    }
…
```

Alternative using a URL parameter to search for an author (optional). Combination of RequestParam and PathVariable is possible



REST with Spring Boot

- **Retrieve all Book resources (GET)**



REST with Spring Boot

- **Add Resource (POST)**

```
@PostMapping("/books")
public Book addBook(@RequestBody Book book) {
    return bookRepository.save(book);
}
```

Using `@PostMapping` indicates that the method expects a POST request.

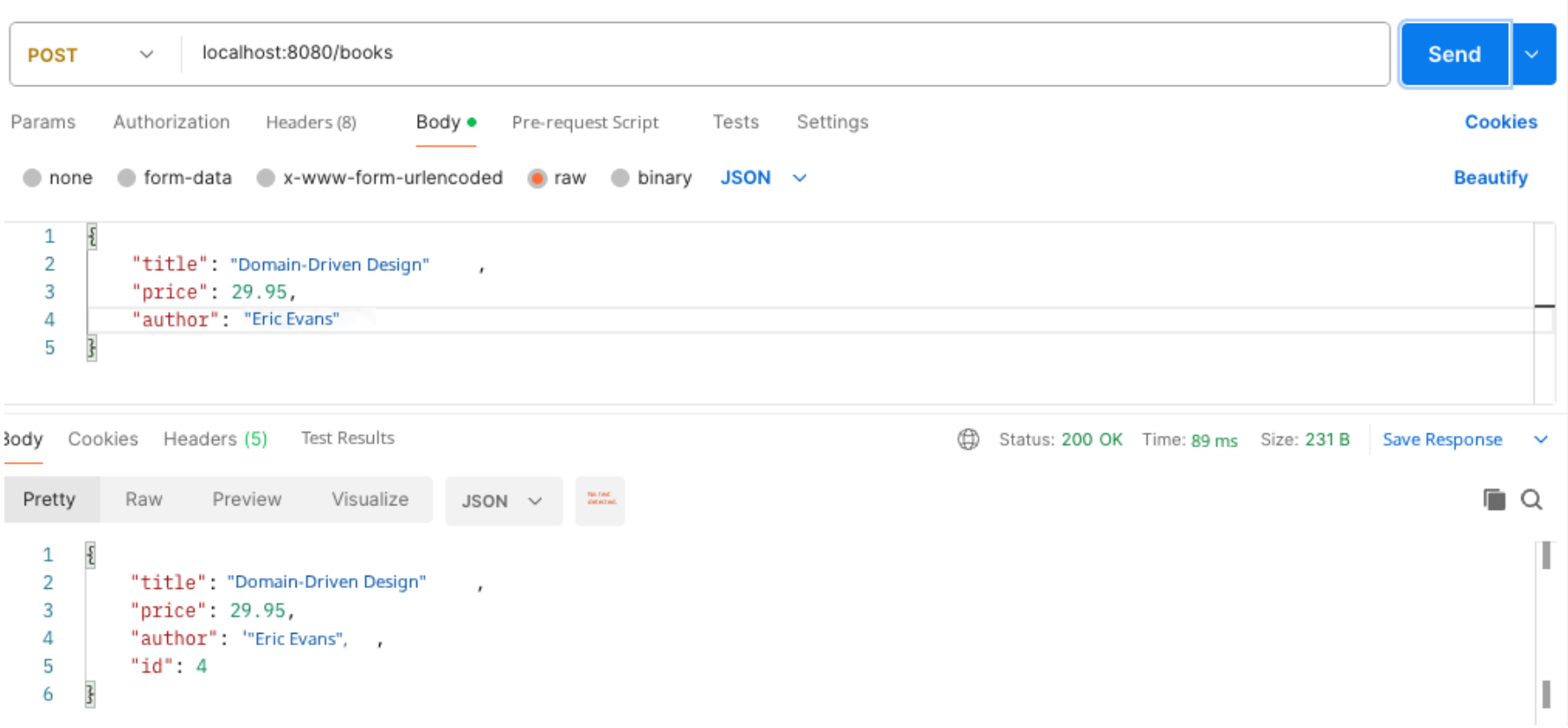
The body of the request is retrieved using a parameter annotated with `@RequestBody` .

As `return` value, the new object is returned to the client.
For new objects, the Location header is typically set with the ID



REST with Spring Boot

- **Add Resource – (POST)**



REST with Spring Boot

- **Delete Resource (DELETE)**

```
@DeleteMapping("/books/{id}")
public Book deleteBook(@PathVariable Long id) {
    Delete Book = bookRepository.findById(id).get();
    bookRepository.deleteById(id);
    return del;
}
```

Using `@DeleteMapping` indicates that the method expects a DELETE request.

The resource is identified by the `@PathVariable id` .

If the resource exists, it is deleted and returned.



REST with Spring Boot

- **Deleting a Resource (DELETE)**

